

docforge Technical Manual

Version 53d0439

2026-09-30

Contents

1 docforge	5
1.1 Purpose	5
1.2 Technologies	5
1.3 Architecture at a Glance	5
1.4 Prerequisites	5
1.5 Local Development	6
1.6 Running Tests	6
1.7 Deployment	6
1.8 Documentation	6
2 Architecture	7
2.1 Overview	7
2.2 Components	7
2.3 Information Flow	8
2.4 External Systems	8
2.5 Infrastructure	8
2.6 Deployment Architecture	8
3 Design	9
3.1 Design Principles	9
3.2 Component Boundaries	9
3.3 Key Abstractions and Patterns	9
3.4 Configuration Strategy	10
3.5 Error Handling	10
3.6 Logging	10
3.7 Authentication and Authorisation	10
3.8 State Management	10
3.9 API Conventions	11
4 How It Works	12
4.1 Workflows	12
5 Data Model	15
5.1 Overview	15
5.2 Entities	15
5.3 Relationships	15
5.4 Constraints and Indexes	15
5.5 Ownership and Retention	16
5.6 Migrations	16
5.7 Business Rules	16

6 API	17
6.1 Overview	17
6.2 Endpoints	17
6.3 Errors	18
7 Security	19
7.1 Trust Boundaries	19
7.2 Authentication	19
7.3 Authorisation	19
7.4 Secrets Management	19
7.5 Session Handling	20
7.6 API Security	20
7.7 Network Exposure	20
7.8 Input Validation	20
7.9 Dependency Security	20
7.10 Logging and Audit	20
7.11 Sensitive Data	20
7.12 Rate Limiting	20
7.13 Encryption and TLS	21
7.14 Security Assumptions	21
7.15 Responsibilities	21
8 Deployment	22
8.1 Environments	22
8.2 Build	22
8.3 Deploy	22
8.4 Configuration	23
8.5 Rollback	23
9 Testing	24
9.1 Strategy	24
9.2 Unit Tests	24
9.3 Integration Tests	24
9.4 End-to-End Tests	24
9.5 Security Tests	24
9.6 Manual and Smoke Tests	25
9.7 Test Environments and Data	25
9.8 Running Tests	25
9.9 Required Before Changes	25
10 Operations	26
10.1 Monitoring	26
10.2 Alerts	26
10.3 Routine Operations	26
10.4 Incident Response	26
11 Maintenance	28
11.1 Starting and Stopping	28
11.2 Dependency Updates	28
11.3 Database Migrations	28
11.4 Secret Rotation	28
11.5 Configuration Management	29

11.6	Logs	29
11.7	Backups	29
11.8	Releases	29
11.9	Making Production Changes	29
11.10	Known Technical Debt	29
11.11	Fragile Areas	29
12	Troubleshooting	31
12.1	Known Problems	31
13	Dependencies	33
13.1	Critical Dependencies	33
14	ADR-001: The agent proposes patches; a human applies them, on a doc-path allowlist	35
14.1	Status	35
14.2	Context	35
14.3	Options Considered	35
14.4	Decision	35
14.5	Rationale	35
14.6	Consequences	36
14.7	Related Components	36
15	ADR-002: Hand-written agent loop with a hard budget guard	37
15.1	Status	37
15.2	Context	37
15.3	Options Considered	37
15.4	Decision	37
15.5	Rationale	37
15.6	Consequences	38
15.7	Related Components	38
16	ADR-003: Run the agent and the PDF build in OpenShell sandboxes	39
16.1	Status	39
16.2	Context	39
16.3	Options Considered	39
16.4	Decision	39
16.5	Rationale	40
16.6	Consequences	40
16.7	Related Components	40
17	ADR-004: CI runs only deterministic checks; the model is never called in CI	41
17.1	Status	41
17.2	Context	41
17.3	Options Considered	41
17.4	Decision	41
17.5	Rationale	41
17.6	Consequences	41
17.7	Related Components	42
18	ADR-005: Detect documentation drift with a configurable impact map and a commit-trailer escape hatch	43

18.1 Status	43
18.2 Context	43
18.3 Options Considered	43
18.4 Decision	43
18.5 Rationale	44
18.6 Consequences	44
18.7 Related Components	44
19 ADR-006: Build the manual with pandoc, a Lua filter, tectonic and mermaid-cli	45
19.1 Status	45
19.2 Context	45
19.3 Options Considered	45
19.4 Decision	45
19.5 Rationale	46
19.6 Consequences	46
19.7 Related Components	46
20 ADR-007: One table defines the documentation set for scaffolding, checking and the agent prompt	47
20.1 Status	47
20.2 Context	47
20.3 Options Considered	47
20.4 Decision	47
20.5 Rationale	47
20.6 Consequences	48
20.7 Related Components	48
21 Changelog	49
21.1 [Unreleased]	49

Chapter 1

docforge

1.1 Purpose

docforge is a command-line tool for keeping engineering documentation in a repository accurate ("docs-as-code"). It scaffolds a fixed documentation set, checks it deterministically (structure, sections, links, ADRs, drift between code and docs), runs an AI agent inside a sandbox to *propose* documentation patches, and builds the Markdown into a versioned PDF manual.

Design background lives in `specification.json` and `definition_of_done.md` (not reviewed while writing these docs: uncertain — verify with developer).

1.2 Technologies

- Python 3.12+, [Typer](#) for the CLI, the anthropic SDK for the agent, built with hatchling and managed with uv.
- External tools for the PDF manual: pandoc, tectonic (LaTeX), mermaid-cli (mmdc).
- OpenShell and Docker for the agent and build sandboxes.
- Dev tooling declared: pytest, ruff, mypy (strict), jsonschema.

See docs/DEPENDENCIES.md.

1.3 Architecture at a Glance

Commands: `init`, `check`, `build`, `agent {impact,adr,bootstrap}`, `apply`, `usage`. The agent can only read the repository and `stage` Markdown writes; a human reviews `docforge.patch` and `docforge apply` commits it to a new branch. CI runs only the deterministic check; the model is never called in CI. Details in docs/ARCHITECTURE.md.

1.4 Prerequisites

Run `./init.sh` (Linux/WSL). It reports, and does not install, missing: python3 3.12+, uv, git, pandoc, tectonic, Node `>= 20`, mmdc, pdftotext (poppler-utils), docker, openshell. It then runs `uv sync`.

1.5 Local Development

```
./init.sh                # check prerequisites + create the venv (default ~/.venvs/docfo
./dev.sh docforge check  # run any command in the dev environment
./dev.sh ruff check src tests
./dev.sh mypy src
```

dev.sh is uv run with UV_PROJECT_ENVIRONMENT pointed at the venv outside the repo. Model calls need ANTHROPIC_API_KEY (or the OpenShell provider); see docs/MAINTENANCE.md.

1.6 Running Tests

```
./dev.sh pytest -q
```

See docs/TESTING.md.

1.7 Deployment

docforge is not a deployed service. It is installed as a Python wheel; CI installs it with uvx from a git URL, and the sandbox images are built by sandbox/build-images.sh. See docs/DEPLOYMENT.md.

1.8 Documentation

- Architecture, Design, How it works
- Data model, API / CLI, Security
- Deployment, Testing, Operations, Maintenance
- Troubleshooting, Dependencies
- Decisions: docs/adr/
- CHANGELOG

Chapter 2

Architecture

Which components exist and how information flows between them.

2.1 Overview

docforge is a single Python package (src/docforge) exposing one CLI. It has no server, database or long-running process. It works on a target git repository (its own, for self-documentation) and on a small set of files there: docforge.toml, docs/, README.md, CHANGELOG.md, documentation/.

2.2 Components

Component	Role
cli.py	Typewriter wiring only; one function per command.
config.py	Loads and strictly validates docforge.toml; defines the
doctypes.py	Table of the documentation set and required H2 sections
init.py	Scaffolds missing files and guesses the impact map from
checks.py, markdown.py, findings.py	Deterministic checks (structure, sections, links, ADRs,
drift.py, git.py, globs.py	Compares code changes against the impact map; thin g
agent/	Model loop (runner.py), tool policy (policy.py), spend
apply.py	Applies a reviewed patch on a new branch, re-checking
build.py, templates/docforge.lua	Markdown to pandoc to LaTeX to PDF; Lua filter rende
sandbox/	OpenShell policies, Docker images and scripts for runn

2.3 Information Flow

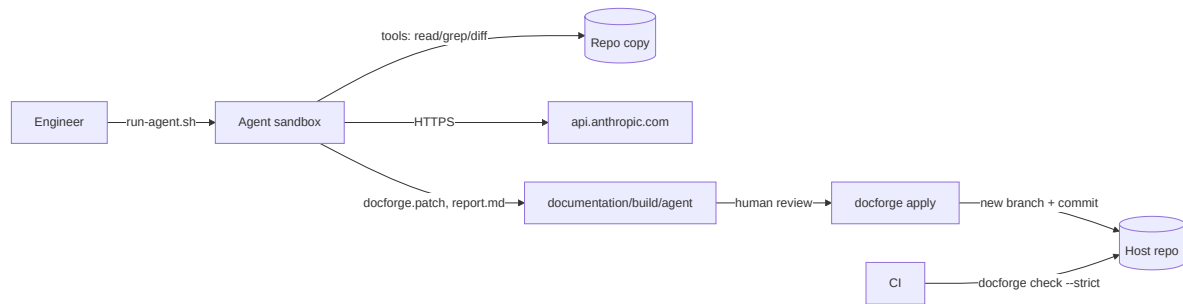


Figure 2.1: Diagram

1. The agent only touches the repository through six tools (`list_files`, `read_file`, `grep`, `git_diff`, `write_doc`, `finish`). Writes are held in memory.
2. At the end, staged writes become a unified diff (`docforge.patch`) plus `report.md`, `trace.log`, `policy.log`, `run.json`.
3. A human reviews, then `docforge apply` checks the paths again and commits on a new branch. Nothing is pushed.

Workflow detail: `HOW_IT_WORKS.md`.

2.4 External Systems

- Anthropic API (`api.anthropic.com:443`), model `claude-sonnet-5-5`, used only by `docforge agent`.
- `git CLI`, `pandoc`, `tectonic`, `mermaid-cli (mmdc)`.
- `OpenShell (sandbox runtime)` and `Docker`.
- `GitHub Actions (CI)`. The workflow installs `docforge` from `github.com/beese54/docforge`; the repository's own status note says that repository did not yet exist (2026-09-30).

2.5 Infrastructure

No hosted infrastructure. Local state is the usage ledger `~/.docforge/usage.jsonl` (override with `DOCFORGE_HOME`) and build output under `documentation/build/` (git-ignored by `init`). Two Docker images, `docforge-agent:<version>` and `docforge-build:<version>`, run under `OpenShell`.

2.6 Deployment Architecture

See `DEPLOYMENT.md`. In short: a wheel built with `uv build`; the agent image installs it with `pip`; CI uses `uvx --from <git url>`.

Chapter 3

Design

Design intent: boundaries, abstractions and conventions, not a file listing.

3.1 Design Principles

- **The model proposes, a human disposes.** The agent never writes to disk in the repository; see ADR-001.
- **Enforce by construction, not by prompt.** Path limits and spend caps are code, not instructions (ADR-002).
- **Deterministic checks are separate from the model.** check never uses the network (ADR-004).
- **Honesty markers.** Unverified claims are written "uncertain — verify with developer" and counted by check.
- **One source of truth for the doc set.** doctypes.py drives scaffolding, checking and the agent prompt.

3.2 Component Boundaries

- cli.py only wires commands; logic sits in per-command modules, imported lazily inside each command.
- All repository access by the agent goes through agent/policy.py:ToolLayer. All git access goes through git.py.
- apply.py runs on the host and does not trust the patch: it re-validates every path.
- Sandboxes: the agent has network only to api.anthropic.com; the PDF build has none (ADR-003).

3.3 Key Abstractions and Patterns

- Config / ImpactRule: the impact map (globs to docs, optionally adr_worthy) (ADR-005).
- Finding with severity (info/warn/error) and a Check = function of Context; checks.run accepts extra checks (drift is injected this way).
- ToolLayer: stages writes in a dict, records denials as PolicyEvents.

- BudgetGuard: sizes `max_tokens` so the worst case fits the remaining allowance; interrupted streams are charged worst-case.
- Escape hatch: a Docs-Impact: none – <reason> commit trailer; an empty reason is reported and ignored.
- Build pipeline: pandoc plus a Lua filter that turns Mermaid blocks into figures and rewrites cross-file links (ADR-006).

3.4 Configuration Strategy

A single `docforge.toml` at the repository root. Unknown keys are errors so typos cannot silently disable a check. Sections: `[project]`, `[docs]`, `[[impact]]`, `[manual]`, `[budget]`, `[uncertain]`. The agent write allowlist (`README.md`, `CHANGELOG.md`, `docs/**`) is deliberately not configurable. Environment: `DOCFORGE_HOME` (ledger location), `ANTHROPIC_API_KEY`, `DOCFORGE_PUPPETEER_CONFIG` (Mermaid). Build defaults can be overridden by `documentation/manual.yaml` and `documentation/templates/manual-header.tex`.

3.5 Error Handling

Each layer raises its own exception (`ConfigError`, `GitError`, `ToolError`, `AgentError`, `BudgetExceeded`, `BuildError`, `ApplyError`) which `cli.py` maps to exit codes: 2 = configuration/environment, 3 = agent failure or missing credentials, 1 = check errors, patch refused or build content failure. Tool denials go back to the model as error results rather than aborting. On any agent failure `docforge.patch` and `report.md` are deleted, `trace.log`, `policy.log`, `run.json` are kept, and finished documents are saved in `docforge.partial.patch`. Stream drops are retried 3 times with backoff.

3.6 Logging

There is no logging framework. Agent runs write `trace.log` (per-turn cost and tool calls), `policy.log` (denials) and `run.json` into the output directory; `spend` is appended to `usage.jsonl`. Other commands print to `stdout/stderr`.

3.7 Authentication and Authorisation

Only outbound: the Anthropic SDK reads credentials from the environment; inside the sandbox a placeholder key is swapped for the real one at egress. There are no users or roles. "Authorisation" is the path policy. See `SECURITY.md`.

3.8 State Management

Stateless apart from the append-only usage ledger and run outputs. The monthly cap is computed from the ledger at run start, so it spans runs, including sandboxed ones (the script copies the ledger in and out).

3.9 API Conventions

The public interface is the CLI (API.md). `--path` selects the repository root; `check` supports `--format json`; `--base` selects the drift baseline.

Chapter 4

How It Works

End-to-end workflows.

4.1 Workflows

4.1.1 Workflow: Scaffold (docforge init)

1. Trigger: engineer runs `docforge init [--path .]`.
2. Received by: `cli.init` then `init.scaffold`.
3. Processing: builds a plan of skeleton docs (from `doctypes.py`), CHANGELOG, ADR template, manual defaults, CI workflow and `docforge.toml`. The impact map is guessed by matching candidate globs against files that exist.
4. Services called: `git ls-files` (falls back to a directory walk).
5. Data stored: only missing files are created (opened with mode `x`); `documentation/build/` is appended to `.gitignore`.
6. Asynchronous work: none.
7. Result: list of created/existing files.
8. What can fail: nothing is overwritten; a guessed impact map may be wrong and must be reviewed.

4.1.2 Workflow: Check (docforge check)

1. Trigger: engineer or CI (`docforge check --strict --base <ref>`).
2. Received by: `cli.check`, `checks.run` plus the drift check.
3. Processing: required files exist; required H2 sections; internal links and anchors (case-exact); ADR naming, numbering (no gaps), title and status; count of "uncertain" markers (limit `[uncertain] max_markers`); drift. `--strict` turns soft rules and drift in `adr_worthy` categories into errors.
4. Services called: `git (log, merge-base, diff)`.
5. Data stored: none.
6. Asynchronous work: none.
7. Result: findings as text or JSON; exit 1 if any error.
8. What can fail: base ref missing (shallow clone) is an error; no base ref or not a repo is only informational; invalid `docforge.toml` exits 2.

Drift: for each impact rule, if a non-overridden changed file matches its globs and none of its docs changed in the same range (or uncommitted), a finding is raised.

4.1.3 Workflow: Agent run (`docforge agent impact|adr|bootstrap`)

1. Trigger: engineer, normally via `sandbox/run-agent.sh <repo> <task>`.
2. Received by: `cli._run_agent`, which loads config, checks git and credentials and creates a BudgetGuard (per-run cap by command, monthly cap).
3. Processing: loop in `agent/runner.py`. Each turn counts input tokens, sizes `max_tokens` to the remaining budget, streams a response, records cost, executes tool calls through `ToolLayer`. Ends when the model calls `finish`, or when it stops without tools (its text becomes the report). Turn limits: `impact` 40, `adr` 20, `bootstrap` 120.
4. Services called: Anthropic API (model `claude-sonnet-5-5`, adaptive thinking, effort medium).
5. Data stored: `usage.jsonl` per call; in the output dir `docforge.patch`, `report.md`, `policy.log`, `trace.log`, `run.json` (default `documentation/build/agent`).
6. Asynchronous work: none; streaming is synchronous.
7. Result: a patch and report for human review. The repository is untouched.
8. What can fail: budget exhausted, model refusal, `max_tokens` hit, turn limit, API or connection errors, missing credentials (exit 3). Denied tool calls are logged and returned to the model.

4.1.4 Workflow: Sandboxed agent run (`sandbox/run-agent.sh`)

1. Trigger: engineer on the host.
2. Processing: copies the usage ledger; creates an OpenShell sandbox from `docforge-agent:<version>` with `agent-policy.yaml` and the anthropic provider; uploads the repository (working tree plus `.git`) as a tar; runs `docforge agent ... --out /sandbox/out`; downloads outputs and the ledger back to `<repo>/documentation/build/agent`; deletes the sandbox on exit.
3. Failure modes: OpenShell/Docker problems; the script propagates the agent's exit status.

4.1.5 Workflow: Apply (`docforge apply <patch>`)

1. Trigger: engineer after reviewing `report.md` and the patch.
2. Processing: extracts all paths in the patch; refuses anything outside `README.md`, `CHANGELOG.md`, `docs/**.md`; names the branch `docforge/<task>-<sha>` from `run.json` (or `--branch`); `git apply --check --index`; creates the branch, applies, commits.
3. Result: commit on a new branch. Never pushes.
4. What can fail: empty patch, disallowed path, existing branch, patch not applying cleanly (exit 1).

4.1.6 Workflow: Build manual (`docforge build`)

1. Trigger: engineer or CI job manual (`workflow_dispatch` or tags).
2. Processing: requires `pandoc` and `tectonic` (`tectonic` not needed for `--format tex`); expands `[manual]` chapters (globs sorted, `000-*` skipped); fills version (`git describe --tags --always --dirty`), date and SHA into the LaTeX header; runs `pandoc` with the Lua filter.
3. Result: `documentation/build/<name>-manual-<version>.pdf`.
4. What can fail: missing tools (exit 2), no chapters (exit 2), `pandoc` failure (exit

1), Mermaid render failure (warning; error with `--strict`). Without `mmdc`, diagrams remain code blocks.

4.1.7 Workflow: Usage report (docforge usage)

Reads the ledger and prints spend per run, this month and total.

Chapter 5

Data Model

Entities, relationships, constraints, ownership and business rules.

5.1 Overview

docforge has no database. Its persistent data is a JSON-lines usage ledger and the files it reads or produces in the target repository.

5.2 Entities

- **UsageRecord** (one JSON object per line in `usage.jsonl`): `ts`, `run_id`, `command`, `model`, `input_tokens`, `output_tokens`, `cache_read_tokens`, `cache_write_tokens`, `cost_usd`, `note`.
- **Config** (`docforge.toml`): project name/title, required docs, impact rules (category, globs, docs, `adr_worthy`), manual chapters, budget, uncertain limit.
- **Finding**: rule, severity, file, message, optional line (in-memory).
- **Run outputs**: `docforge.patch`, `docforge.partial.patch`, `report.md`, `policy.log`, `trace.log`, `run.json` (`run_id`, `command`, `status`, `cost_usd`, `turns`, `files`).
- **ADR files**: `docs/adr/NNN-kebab-title.md`.

5.3 Relationships

Records sharing a `run_id` belong to one agent run; `run.json` sits beside the patch and is read by `apply` to name the branch. Impact rules map file globs to doc paths.

5.4 Constraints and Indexes

Config: unknown keys rejected; category names unique; budgets positive numbers; `max_markers` non-negative integer. ADRs: 3-digit number, no duplicates, no gaps, valid status. No indexes.

5.5 Ownership and Retention

The ledger is per-user, at `$DOCFORGE_HOME` or `~/.docforge`, append-only; no rotation or retention policy exists in code. Run outputs default to `documentation/build/agent` (git-ignored via `documentation/build/`).

5.6 Migrations

None. The ledger format has no version field; a changed `UsageRecord` would break reading older lines (`UsageRecord(**json)`), so add fields with defaults.

5.7 Business Rules

- Monthly spend is the sum of ledger records whose timestamp starts with the current `YYYY-MM`.
- Allowance = $\min(\text{per-run cap} - \text{run spent}, \text{monthly cap} - \text{month spent before run} - \text{run spent})$.
- Default caps: impact \$0.50, adr \$0.30, bootstrap \$3.00 per run; \$10.00 per month.
- Interrupted calls are recorded at worst-case cost.

Chapter 6

API

Behaviour, assumptions and side effects that a generated API spec does not capture.

6.1 Overview

The interface is the docforge CLI (Typer). There is no HTTP API. `--path` (repository root, default `.`) applies to all commands except usage. `docforge --version` prints the version.

6.2 Endpoints

6.2.1 docforge init

- Purpose: create missing docs, config, CI workflow, ADR template.
- Side effects: writes files (never overwrites); may append to `.gitignore`.

6.2.2 docforge check [--strict] [--format text|json] [--base REF]

- Purpose: run deterministic checks and drift detection.
- Side effects: none. Exit 0/1 (errors)/2 (bad config). Default base: merge-base with origin/main, origin/master, main or master.

6.2.3 docforge build [--out DIR] [--format pdf|tex] [--strict]

- Purpose: build the PDF (or LaTeX) manual into documentation/build.
- Side effects: writes output and a temp dir. Exit 2 for environment problems, 1 for content.

6.2.4 docforge agent impact [--base REF] [--out DIR], agent adr TITLE [--note TEXT], agent bootstrap

- Purpose: run the model to propose doc changes.
- Authentication: Anthropic credentials from the environment.
- Side effects: paid API calls; appends to the usage ledger; writes outputs to documentation/build/agent (or `--out`). Never modifies tracked docs. Requires a git repository. Exit 3 on agent failure or missing credentials.

6.2.5 docforge apply PATCH [--branch NAME]

- Purpose: apply a reviewed patch.
- Side effects: creates a branch (default docforge/<task>-<sha>), stages and commits. Never pushes. Leaves the new branch checked out.

6.2.6 docforge usage [--last]

- Purpose: show model spend from the ledger. Side effects: none.

6.3 Errors

Exit codes: 0 success; 1 check errors, refused/failed apply, or build content failure; 2 configuration, environment, bad base ref or non-git repository; 3 agent failure or missing credentials. Messages go to stderr prefixed error:. Note check returns 1 only for error-severity findings; warnings do not fail it unless --strict promotes them.

Chapter 7

Security

Security architecture, trust boundaries and who is responsible for what.

7.1 Trust Boundaries

1. **Model output is untrusted.** It can only act through ToolLayer; writes are staged, not applied.
2. **Agent patch is untrusted at apply time.** docforge apply re-validates every path.
3. **Sandbox vs host.** The agent runs on a copy of the repository in an OpenShell sandbox; only /sandbox/out returns.
4. **Repository content is untrusted input to the model** (prompt injection via file contents is possible; mitigated only by the write allowlist and human review).

7.2 Authentication

No user authentication. Outbound: Anthropic API key via the SDK (ANTHROPIC_API_KEY). In the sandbox the key is a placeholder replaced by OpenShell at egress, only for api.anthropic.com, only for listed binaries (curl, python3).

7.3 Authorisation

Write allowlist README.md, CHANGELOG.md, docs/**, and only .md files; not configurable (ADR-003 and ADR-001). Reads are confined to the repository root: absolute paths, . . . , backslashes, NUL and symlink escapes are denied.

7.4 Secrets Management

Secret-like files are unreadable by the agent and excluded from listings, grep and diff: .env*, *.pem, *.key, id_rsa*, *.p12, .git/**, .npmrc, .pypirc. The real API key is not baked into images or put in the sandbox. Note the .git/** exclusion covers file reads, while the sandbox upload does include .git so the agent can diff.

7.5 Session Handling

Not applicable (no sessions).

7.6 API Security

No inbound API. The CLI's outbound call goes to the Anthropic API over HTTPS.

7.7 Network Exposure

Agent sandbox: `network_policies` is empty; the only egress comes from the anthropic provider profile (`api.anthropic.com:443`). Build sandbox: no egress and no provider. No listening ports.

7.8 Input Validation

- Config: strict key and type validation.
- Tool arguments: strict JSON schemas (`additionalProperties: false`), size limits (200,000-byte reads, 150,000-char diff, 200 grep matches, 2,000 listed files), binary files refused.
- Patch paths: diff headers, `---/+++` and `rename/copy` lines are all parsed for allowlist checking.

7.9 Dependency Security

No scanning is configured in the repository (uncertain — verify with developer). Versions are locked in `uv.lock`. `anthropic` is unpinned in `pyproject.toml`. See `DEPENDENCIES.md`.

7.10 Logging and Audit

Each denial is logged with timestamp, tool, path and reason to `policy.log`; every model call is appended to the usage ledger. Neither is tamper-protected.

7.11 Sensitive Data

Repository contents that the agent reads (excluding secret patterns) are sent to the Anthropic API. Do not run the agent on repositories whose non-secret files may not leave your environment. Secrets under other filenames would not be filtered (uncertain — verify with developer whether additional patterns are needed).

7.12 Rate Limiting

None on requests. Spend is capped per run and per month by construction (ADR-002), plus turn limits.

7.13 Encryption and TLS

Outbound TLS is handled by the Anthropic SDK; the CI installer uses `curl --proto '=https' --tlsv1.2`. The usage ledger is stored unencrypted.

7.14 Security Assumptions

- OpenShell enforces the filesystem, landlock and egress policies (landlock `best_effort`, so weaker kernels may enforce less).
- The host user reviews patches before applying.
- `sandbox/redteam.sh` is the evidence for controls 5.1–5.4b; per the 2026-09-30 status note, the build-sandbox checks still needed a rerun.

7.15 Responsibilities

- Developer running the agent: review `report.md` and the patch, keep the API key on the host, keep spend caps sensible.
- Maintainer: keep policy files, `redteam.sh` and this document in step (strict impact rule security).
- Pilot finding: a downstream project had unused sanitising code; docforge itself is not affected.

Chapter 8

Deployment

How the system is built, configured and deployed, and how to roll back.

8.1 Environments

- **Developer machine** (Linux/WSL): venv managed by uv, default `~/ .venvs/docforge` (README).
- **CI**: GitHub Actions on `ubuntu-24.04`, workflow `docforge`.
- **Sandboxes**: OpenShell sandboxes from two Docker images (`agent`, `build`).
- Staging/production: not applicable; there is no hosted service (uncertain — verify with developer whether any distribution channel such as PyPI is planned).

8.2 Build

- Wheel: `uv build --wheel` (hatchling; package `src/docforge`). Version is in `pyproject.toml` (0.1.0).
- Images: `sandbox/build-images.sh` removes `dist/`, builds the wheel, then builds `docforge-agent:<version>` (`python:3.12-slim`, `git`, `curl`, the wheel, non-root sandbox user) and `docforge-build:<version>` (`ubuntu:24.04`, `pandoc`, `tectonic`, `Node 22`, `mermaid-cli 11`, `headless Chrome`, and a warm-up build of `sandbox/warmup` so builds work offline).
- The build image downloads `tectonic` via an install script from `dropsh.fullyjustified.net` at build time, without a checksum (Node is checksum-verified).

8.3 Deploy

There is nothing to deploy to a server.

- **CI check**: on pull requests and pushes to `main/master`, `uvx --from "$DOCFORGE_SPEC" docforge check --strict --base <ref>`. `DOCFORGE_SPEC` is `docforge @ git+https://github.com/beese54/docforge`; per the 2026-09-30 status note this repository did not exist yet, so the check cannot pass on GitHub until it does (or the variable is changed).

- **CI manual job:** on manual dispatch or tag refs, installs pandoc, tectonic and mermaid-cli, runs `docforge build`, uploads `documentation/build/*.pdf` as artifact `manual`.
- **Agent:** `sandbox/run-agent.sh` (needs the images, the imported anthropic provider profile and an OpenShell gateway). Setup of the profile: see the header of `sandbox/anthropic-profile.yaml`. `sandbox/install-native-docker.sh` installs a separate Docker engine for OpenShell (not read: uncertain — verify with developer what it changes).

8.4 Configuration

See `DESIGN.md` for `docforge.toml`. Deployment-relevant settings: `DOCFORGE_SPEC` (CI), `DOCFORGE_HOME`, `ANTHROPIC_API_KEY` (agent), image tags derived from `pyproject.toml` version. The build sandbox needs `XDG_CACHE_HOME=/opt/cache`, `PUPPETEER_CACHE_DIR=/opt/cache/puppeteer`, `DOCFORGE_PUPPETEER_CONFIG=/opt/docforge/puppeteer` passed explicitly because OpenShell exec sessions do not inherit image `ENV`.

8.5 Rollback

No release process is defined in the repository (uncertain — verify with developer). Practical options: pin `DOCFORGE_SPEC` to an earlier git tag or commit; rebuild images from an earlier commit (image tags equal the version, so rebuilding overwrites a tag unless the version is bumped). Doc patches applied by `docforge apply` are ordinary commits on their own branch and can be dropped by deleting the branch or reverting.

Chapter 9

Testing

Testing strategy and what must pass before a change is accepted.

9.1 Strategy

pytest suite under tests/, one file per module area. Tests use temporary git repositories and an isolated usage ledger (autouse fixture sets DOCFORGE_HOME), so they never touch ~/.docforge. Static checks: ruff (line length 120, rules E,F,W,I,B,UP,SIM) and mypy in strict mode. Test contents were not read in detail beyond conftest.py and test_smoke.py; file names indicate coverage of agent, apply, build, checks, config, drift, init and templates (uncertain — verify with developer what each covers).

9.2 Unit Tests

tests/test_config.py, test_checks.py, test_drift.py, test_init.py, test_templates.py, test_build.py, test_apply.py, test_agent.py, test_smoke.py (CLI --version). The runner accepts a fake client (see run(client: Any ...) comment), which suggests the agent is tested without the network.

9.3 Integration Tests

The repo fixture creates a real git repository, so git-dependent code is exercised with the real git CLI. No dedicated integration suite is defined.

9.4 End-to-End Tests

None automated. Manual pilot runs are recorded in evidence/ (not reviewed here).

9.5 Security Tests

sandbox/redteam.sh runs live attacks against the OpenShell sandboxes: key absence (5.1), key swap works only for Anthropic (5.1b), blocked egress (5.2, 5.2b), read-only system paths (5.3, 5.3b), build sandbox offline (5.4, 5.4b). Needs docker,

openshell and images; the only API request is the free GET /v1/models. `--agent-only` skips build-sandbox checks.

9.6 Manual and Smoke Tests

`docforge check --strict` on this repository; `docforge build --format tex` to inspect LaTeX; `docforge build --strict` to ensure Mermaid renders (the image build does this with sandbox/warmup).

9.7 Test Environments and Data

Local WSL/Linux via `./init.sh`. `sandbox/warmup/` is a fixture manual. `git` must be installed.

9.8 Running Tests

```
./dev.sh pytest -q
./dev.sh ruff check .
./dev.sh mypy src
sandbox/redteam.sh --agent-only # needs images + OpenShell
```

9.9 Required Before Changes

- `pytest` passes; `ruff` and `mypy` clean (no CI job enforces this: see MAINTENANCE.md).
- `docforge check --strict` passes (the only thing CI enforces).
- Changes to `policy/budget/apply/sandbox` files: rerun `sandbox/redteam.sh`.

Chapter 10

Operations

Running the system day to day: monitoring, alerts, routine tasks, incidents.

10.1 Monitoring

No service to monitor. Monitor spend with docforge usage (per run, this month, total) and CI status of the docforge workflow. Agent health per run: `run.json` status and `trace.log`.

10.2 Alerts

None configured. The monthly cap (default \$10) is the only automatic guard; it refuses further calls rather than alerting.

10.3 Routine Operations

- Run an impact check after significant code changes: `sandbox/run-agent.sh <repo> impact --base <ref>`, review `report.md`, then docforge apply `documentation/build/agent/docforge.patch --path <repo>`, push manually.
- First-time pass on a repository: `sandbox/run-agent.sh <repo> bootstrap` (default cap \$3.00).
- Draft an ADR: `sandbox/run-agent.sh <repo> adr "Title" --note "..."`.
- Rebuild sandbox images after a version bump: `sandbox/build-images.sh`.
- Re-verify sandbox controls: `sandbox/redteam.sh`.
- Build the manual: docforge build or the CI manual job.

10.4 Incident Response

- Suspected key leak: rotate the Anthropic key at the provider and update the OpenShell provider (uncertain — verify with developer for commands). Check `sandbox/redteam.sh` control 5.1.
- Unexpected spend: docforge usage; inspect `~/.docforge/usage.jsonl` (note field marks refusals and worst-case charges).

- Agent attempted disallowed writes or reads: see `policy.log`; the denials are already enforced, but review the patch carefully.
- A bad patch was applied: it is on its own branch; delete the branch.
- Other symptoms: `TROUBLESHOOTING.md`.

Chapter 11

Maintenance

Written for the engineer who inherits this system.

11.1 Starting and Stopping

There are no services. Agent runs are short-lived processes; sandboxes are deleted by the script's exit trap. If a run is killed, leftover sandboxes named `dfa-<timestamp>` may remain: list and delete them with the `openshell sandbox` commands (exact list command: uncertain — verify with developer).

11.2 Dependency Updates

Python dependencies: `typer>=0.12` and `anthropic` (unpinned) in `pyproject.toml`, locked in `uv.lock`. Update with `uv`, then run the tests and `docforge check`. Changing `pyproject.toml/uv.lock` triggers the strict dependencies drift rule, so update `DEPENDENCIES.md` in the same change. Model name and prices are hard-coded in `agent/budget.py`; update them together when changing the model. Sandbox images pin Node major 22, `mermaid-cli 11`, Ubuntu 24.04 and python 3.12; `tectonic` is unpinned.

11.3 Database Migrations

Not applicable: no database. See `DATA_MODEL.md`.

11.4 Secret Rotation

The only secret is the Anthropic API key. On the host it is held by the OpenShell `anthropic` provider (created from `ANTHROPIC_API_KEY`; exact rotation command: uncertain — verify with developer). It is never placed in the sandbox. See `SECURITY.md`.

11.5 Configuration Management

`docforge.toml` is version-controlled and reviewed like code. After changing the impact map, run `docforge check --strict`. The map in this repository was guessed by `docforge init` and flagged in its comment for review.

11.6 Logs

Per agent run: `documentation/build/agent/{trace.log,policy.log,run.jsonl}`.
Spend: `~/.docforge/usage.jsonl` or `docforge usage`. OpenShell denials: `open-shell logs <sandbox>` (used by `redteam.sh`).

11.7 Backups

Nothing to back up except the usage ledger `~/.docforge/usage.jsonl`, which enforces the monthly cap; losing it resets the month's tally to zero.

11.8 Releases

No release automation. The manual build job runs on tag refs and the version string comes from `git describe`. Package version is `0.1.0`. Bumping it changes the image tags used by `run-agent.sh` and `redteam.sh`.

11.9 Making Production Changes

Not a service. For changes: run tests and lint (`TESTING.md`); if you touch policy, budget, apply or sandbox files run `sandbox/redteam.sh`; add a Docs-Impact trailer or update the mapped docs.

11.10 Known Technical Debt

- CI installs docforge from a repository that did not exist at last status (2026-09-30).
- The build-sandbox red-team checks (5.4, 5.4b) still needed a rerun under OpenShell at that date.
- No CI job runs `pytest`, `ruff` or `mypy` (workflow only runs `docforge check` and the manual build).
- Prices and model id are hard-coded.
- Agent impact runs did not add sources comments to CHANGELOG entries (status note).

11.11 Fragile Areas

11.11.1 Fragile area: Policy enforcement point

- Component: `agent/policy.py`, `apply.py`, `config.AGENT_WRITABLE`
- Why it is sensitive: the only barrier between model output and the repository.

- What depends on it: the guarantee that agent patches only touch docs.
- What must be tested before changing it: `tests/test_agent.py`, `tests/test_apply.py`, `sandbox/redteam.sh`.
- Relevant files: `src/docforge/agent/policy.py`, `src/docforge/apply.py`
- Relevant ADR: ADR-001

11.11.2 Fragile area: Budget guard

- Component: `agent/budget.py`
- Why it is sensitive: hard-coded prices; wrong values silently defeat the cap.
- What must be tested before changing it: `tests/test_agent.py`.
- Relevant ADR: ADR-002

11.11.3 Fragile area: Streaming retry in the runner

- Component: `agent/runner.py`
- Why it is sensitive: it imports `httpx2`, described as the SDK's HTTP client, but `httpx2` is not a direct dependency in `pyproject.toml` (uncertain — verify with developer). A dropped stream is retried only for connection/transport errors.

11.11.4 Fragile area: PDF build environment

- Component: `build.py`, `docforge.lua`, `sandbox/build.Dockerfile`
- Why it is sensitive: depends on `pandoc/tectonic/Chromium`; Chromium needs `--no-zygote` under OpenShell `seccomp`; `exec` sessions do not inherit image ENV.
- Relevant ADR: ADR-006

Chapter 12

Troubleshooting

Practical guide. Keep historical lessons even after the bug is fixed.

12.1 Known Problems

12.1.1 Problem: Agent run fails with "budget exhausted"

- Possible symptoms: exit 3, message with per-run and monthly caps.
- Likely causes: the guard refuses a call when the remaining allowance cannot fund at least 2,048 output tokens after worst-case input cost.
- Diagnostic steps: `docforge usage`; check `[budget]` in `docforge.toml`.
- Resolution: raise `[budget.per_run_usd]` or `monthly_usd`, or wait for next month.
- Relevant logs: `trace.log`, `~/.docforge/usage.jsonl` (refusals are recorded with a note).
- Relevant components: `agent/budget.py`.
- Escalation considerations: none.

12.1.2 Problem: Agent run stops without producing `docforge.patch`

- Possible symptoms: exit 3; `run.json` status failed.
- Likely causes (all seen in the pilot's early runs per the status note): a single response too long (`max_tokens`), the turn limit reached, a network stream dropped mid-reply. Also model refusal.
- Diagnostic steps: read `trace.log` (ends with `FAILED:`).
- Resolution: finished documents are kept in `docforge.partial.patch`; review it and apply manually if acceptable, or rerun. Dropped streams are already retried 3 times.
- Relevant components: `agent/runner.py`.

12.1.3 Problem: "no Anthropic credentials"

- Likely causes: `ANTHROPIC_API_KEY` unset; or the OpenShell anthropic provider not attached/imported.
- Resolution: set the key on the host for direct runs; for the sandbox import `sandbox/anthropic-profile.yaml` and create the provider.

12.1.4 Problem: docforge apply refuses the patch

- Likely causes: patch touches a path outside README.md, CHANGELOG.md, docs/**/*.md; branch already exists; patch does not apply cleanly because the docs changed since the run.
- Resolution: rerun the agent on the current tree, or pass --branch.

12.1.5 Problem: docforge check reports drift or "base ref not found"

- Likely causes: a code file in an impact category changed without its docs; or a shallow CI clone.
- Resolution: update the mapped docs, or add a commit trailer Docs-Impact: none – <reason> (a reason is mandatory). CI uses fetch-depth: 0 for this reason.

12.1.6 Problem: PDF build fails or diagrams stay as code

- Symptoms: "missing tools", or warning "mmdc not found", or DOCFORGE-MERMAID-FAILED.
- Resolution: install pandoc, tectonic, mermaid-cli (./init.sh reports which). Mermaid needs Node >= 20 and headless Chrome. In containers set DOCFORGE_PUPPETEER_CONFIG. Use --format tex to inspect the LaTeX.
- Historical lessons (from build/sandbox comments): under OpenShell, Chromium's zygote needs a user namespace which seccomp forbids, hence --no-zygote; exec sessions do not inherit image ENV, so tectonic tries the (blocked) network unless XDG_CACHE_HOME is passed; uploads from a Windows drive (/mnt/c) arrive as zero bytes, so stage on the Linux filesystem.

12.1.7 Problem: sandbox cannot reach the OpenShell gateway

- Likely cause (status note): Docker Desktop's network cannot reach the Open-Shell gateway.
- Resolution: the status note says a separate native Docker engine was installed with sandbox/install-native-docker.sh; also wsl --update was needed. Details: uncertain — verify with developer.

12.1.8 Problem: CI docforge check fails at install

- Likely cause: DOCFORGE_SPEC points at github.com/beese54/docforge, which did not exist at 2026-09-30.
- Resolution: publish the repository or change DOCFORGE_SPEC.

Chapter 13

Dependencies

Dependencies that would be hard to replace or could significantly affect the application.

13.1 Critical Dependencies

13.1.1 Anthropic API and anthropic SDK

- Purpose: model calls for the agent.
- Current version: SDK unpinned in `pyproject.toml`; resolved version is in `uv.lock` (not read). Model `claude-sonnet-5-5`, hard-coded.
- Why it is used: the only model provider implemented.
- Replacement difficulty: high. The runner uses streaming, `count_tokens`, adaptive thinking, `output_config`, prompt caching and strict tool schemas; prices are hard-coded per model in `budget.py`.
- Upgrade considerations: the runner imports `httpx2` (the SDK's HTTP client) to catch a stream cut mid-body (`httpx2.RemoteProtocolError`, which the SDK does not wrap). `httpx2` is therefore declared as a direct dependency in `pyproject.toml`, so an SDK change of HTTP client cannot silently break that import. The runner also catches `TypeError` for missing credentials. Both depend on SDK internals, so re-run `tests/test_agent.py` after any SDK upgrade.
- Known limitations: costs come from a price table copied on 2026-09-25 (per its comment); it may go stale.

13.1.2 OpenShell

- Purpose: sandbox runtime enforcing filesystem and egress policy for the agent and the build.
- Current version: 0.1.x (per `anthropic-profile.yaml`, which notes it ships no profiles).
- Replacement difficulty: high; policy files and `run-agent.sh` use its CLI. See ADR-003.
- Known limitations: needs a Docker engine reachable by its gateway; `landlock` is `best_effort`.

13.1.3 **pandoc, tectonic, mermaid-cli**

- Purpose: PDF manual. pandoc 3.1.3 from Ubuntu 24.04 in the build image; tectonic unpinned; mermaid-cli 11; Node 22.
- Replacement difficulty: medium; the Lua filter is pandoc-specific. See ADR-006.

13.1.4 **Typer**

- Purpose: CLI framework (typer $\geq$ 0.12). Low-to-medium replacement effort; only cli.py uses it.

13.1.5 **git CLI**

- Purpose: drift detection, diffing, apply and versioning, all through git.py. Assumed installed; a missing git raises GitError.

13.1.6 **uv / hatchling**

- Purpose: environment and build (uv sync, uv build --wheel); CI uses uvx.

Chapter 14

ADR-001: The agent proposes patches; a human applies them, on a doc-path allowlist

14.1 Status

Accepted

14.2 Context

An LLM agent that edits a repository directly could change code or leak content through file writes. The tool is meant to maintain documentation only.

14.3 Options Considered

- Let the agent write files directly.
- Let the agent write only docs, directly.
- Stage writes in memory, output a patch, require a human to apply it (chosen).

14.4 Decision

`ToolLayer.write_doc` only stages content in memory and only for `README.md`, `CHANGELOG.md`, `docs/**` Markdown files. The runner turns staged content into `docforge.patch`. `docforge apply` re-checks every path in the patch, commits on a new branch and never pushes. The allowlist `AGENT_WRITABLE` is deliberately not configurable.

14.5 Rationale

Evidence: comments in `agent/policy.py` ("Nothing here writes to disk"), `apply.py` ("a patch that was edited, or produced by something other than the sandboxed agent, still cannot touch code") and `config.py` ("Deliberately not configurable"). Broader rationale beyond these comments: uncertain — verify with developer.

14.6 Consequences

- Safe by default; every change gets human review.
- Extra step for the user; patches can fail to apply if docs change after the run.
- Code files can never be updated by the agent.

14.7 Related Components

src/docforge/agent/policy.py, src/docforge/apply.py, src/docforge/config.py, SECURITY.md

Chapter 15

ADR-002: Hand-written agent loop with a hard budget guard

15.1 Status

Accepted

15.2 Context

Model calls cost money. Spend must be capped per run and per month without relying on the model's cooperation.

15.3 Options Considered

- The SDK's tool runner.
- A manual loop with a pre-call guard (chosen).

15.4 Decision

agent/runner.py runs the loop manually so that every call passes through BudgetGuard. Before each call the guard counts input tokens and sets max_tokens so the worst case (all input at cache-write rate, all output used) fits the remaining allowance; if under 2,048 tokens fit, the call is refused. Interrupted streams are charged worst-case. Per-command turn limits (impact 40, adr 20, bootstrap 120). Spend goes to ~/.docforge/usage.jsonl.

15.5 Rationale

Stated in the module docstrings: "Manual (not the SDK tool runner) so every call passes through the budget guard first" and "Hard spend caps, enforced by construction".

15.6 Consequences

- The cap cannot be exceeded by a single call.
- Prices and model id are hard-coded; a stale price table weakens the guarantee.
- The ledger over-states rather than under-states interrupted calls.

15.7 Related Components

`src/docforge/agent/runner.py`, `src/docforge/agent/budget.py`, `docforge.toml`
[budget]

Chapter 16

ADR-003: Run the agent and the PDF build in OpenShell sandboxes

16.1 Status

Accepted

16.2 Context

The agent reads a repository and talks to an external API; the PDF build runs pandoc, LaTeX and headless Chromium over repository content. Both should have minimal privileges, and the API key should not be exposed.

16.3 Options Considered

- Run on the host.
- Plain Docker. (The build was verified offline this way per build-policy.yaml.)
- OpenShell sandboxes with policy files (chosen).

16.4 Decision

- Agent sandbox: image docforge-agent, agent-policy.yaml (read-only system paths, writable /sandbox, /tmp; empty network policy), plus the anthropic provider profile granting api.anthropic.com:443 to listed binaries, with the real key injected at egress.
- Build sandbox: image docforge-build with all downloads baked in at image build time, build-policy.yaml with no network at all.
- The repository is uploaded as a copy; only /sandbox/out is returned.
- Controls are verified by sandbox/redteam.sh.

16.5 Rationale

From the policy file comments: writes in the sandbox cannot reach the host repo; the key never enters the sandbox; the build needs no egress. Why OpenShell over alternatives: uncertain — verify with developer.

16.6 Consequences

- Requires OpenShell and a compatible Docker engine (status note: Docker Desktop's network could not reach the gateway).
- Chromium must run with `--no-zygote`; exec sessions do not inherit image ENV.
- Landlock is `best_effort`.

16.7 Related Components

sandbox/, SECURITY.md, DEPLOYMENT.md

Chapter 17

ADR-004: CI runs only deterministic checks; the model is never called in CI

17.1 Status

Accepted

17.2 Context

Documentation quality must be enforced on every change, but model calls are costly, non-deterministic and need credentials.

17.3 Options Considered

- Run the agent in CI.
- Run only deterministic checks in CI; run the agent on demand (chosen).

17.4 Decision

`docforge check` never calls the network or model. CI runs `docforge check --strict`. The agent runs on demand inside a sandbox.

17.5 Rationale

Workflow header: "The model is never called in CI (the agent runs on demand in a sandbox)"; `checks.py` docstring: "Never calls the network or the model."

17.6 Consequences

- CI is free and reproducible; `--strict` makes soft rules and `adr_worthy` drift fail the build.

- Semantic accuracy of docs is not verified by CI; only structure, links, ADR format, markers and drift.

17.7 Related Components

src/docforge/checks.py, .github/workflows/docforge.yml, TESTING.md

Chapter 18

ADR-005: Detect documentation drift with a configurable impact map and a commit-trailer escape hatch

18.1 Status

Accepted

18.2 Context

Docs go stale when code changes without them. A tool needs a cheap, deterministic signal.

18.3 Options Considered

- Rely on reviewers.
- Map code globs to docs and flag changes with no matching doc change (chosen).

18.4 Decision

`docforge.toml` holds `[[impact]]` rules (category, globs, docs, `adr_worthy`). `docforge check` inspects commits in `base..HEAD` plus uncommitted files; if a rule's globs match and none of its docs changed, it warns, or errors under `--strict` when `adr_worthy`. A commit can opt out with `Docs-Impact: none - <reason>`; a missing reason is reported and ignored. `docforge init` guesses the initial map from files present.

18.5 Rationale

Module docstring of `drift.py`; the comment in `docforge.toml`. Choice of globs over other techniques: uncertain — verify with developer.

18.6 Consequences

- The map must be maintained; the current one was guessed and needs review.
- Drift shows only that a doc file changed, not that it is correct.
- Needs full git history in CI (`fetch-depth: 0`).

18.7 Related Components

`src/docforge/drift.py`, `src/docforge/init.py`, `docforge.toml`

Chapter 19

ADR-006: Build the manual with pandoc, a Lua filter, tectonic and mermaid-cli

19.1 Status

Accepted

19.2 Context

Repository Markdown must become a versioned PDF manual, including Mermaid diagrams and cross-file links that make sense on paper.

19.3 Options Considered

Recorded in `tasks/docs-as-code-requirements.md` (question #3) and the T2/T6 commits:

- **Quarto.** Markdown → LaTeX → PDF with Mermaid, cross-references and a preview built in. Fastest to ship, but it brings its own project format and a large toolchain, and gives less control over how multi-file links are resolved.
- **Pandoc with a fully custom LaTeX template (Eisvogel-style).** Maximum control of the look, but a large template to maintain.
- **Pandoc with its default LaTeX template, plus a small include-in-header file and a Lua filter** (chosen).
- **TeX engines:** a full TeX Live install (several GB, needs downloads at build time) or **tectonic** (single binary, fetches only the packages a document uses, and its cache can be pre-warmed for offline builds).

19.4 Decision

`docforge build` runs `pandoc` over the configured chapters with `--defaults`, a generated header and `docforge.lua`, producing LaTeX and a PDF via `tectonic`. The filter renders Mermaid blocks with `mmdc` into numbered figures, and rewrites links:

to files in the manual become internal links, other repo paths become plain text. The title carries git describe version, date and SHA. Defaults can be overridden by files in documentation/.

19.5 Rationale

- The source brief asked for Pandoc and for Markdown to remain the single source of truth. Pandoc reads the same GitHub-flavoured Markdown the repository renders, so no second format is introduced.
- The default template plus a ~20-line header (manual-header.tex) already gives TOC, numbered sections, figures, headers and footers. A custom template would add maintenance without changing the result (deviation from the original spec, recorded in commit cfd3dca).
- A Lua filter is needed anyway: pandoc does not rewrite links whose path does not literally match an input, and relative repo paths mean nothing on paper (build.py, docforge.lua).
- tectonic keeps the build image small and, with its cache warmed at image build time, lets the manual build with **no network**, which is what the build sandbox policy requires (sandbox/build-policy.yaml, red-team 5.4b).

19.6 Consequences

- Requires pandoc, tectonic, mermaid-cli, Chromium; build image bakes them in for offline use (ADR-003).
- Diagram failures are warnings unless --strict.

19.7 Related Components

src/docforge/build.py, src/docforge/templates/docforge.lua, sandbox/build.Dockerfile

Chapter 20

ADR-007: One table defines the documentation set for scaffolding, checking and the agent prompt

20.1 Status

Accepted

20.2 Context

The scaffold, the checker and the agent's instructions must agree on which documents and sections exist.

20.3 Options Considered

- Separate lists per feature.
- A single table in `doctypes.py` (chosen).

20.4 Decision

`doctypes.DOC_TYPES` lists each document, purpose and required H2 sections. `init` renders skeletons from it, `check_sections` verifies against it, and `agent/prompts.py` builds the system prompt from it. The "uncertain" marker text is defined there too.

20.5 Rationale

Module docstring: "so the two can never drift apart".

20.6 Consequences

- Changing a section name updates all three consumers, but existing docs must be updated too or check fails.
- Custom documents beyond the set are only checked by link/uncertain rules (via docs/**).

20.7 Related Components

src/docforge/doctypes.py, src/docforge/init.py, src/docforge/checks.py,
src/docforge/agent/prompts.py

Chapter 21

Changelog

All meaningful changes to this project are recorded here. Format follows [Keep a Changelog](#). Git history is not a substitute.

21.1 [Unreleased]

21.1.1 Added

- Bootstrapped the documentation set: README, the 12 docs/*.md files and ADRs 001-007.

21.1.2 Changed

- README rewritten from a two-line stub into the eight required sections; no deep material existed to move.

21.1.3 Security

21.1.4 Removed